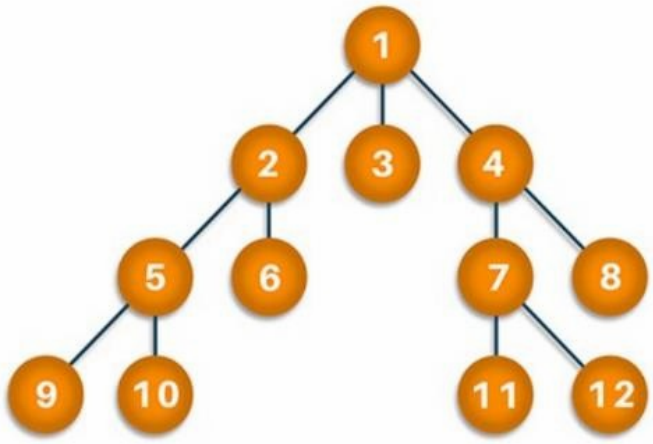
Task 1:

Write a python program to implement DFS and BFS on the graph given below. Consider this to be a directed graph. Start state =1 and Goal state = 11. Your program should stop once Goal state is reached.



BFS:

Code Snapshot:

```python
myGraph={
    "1":["2","3","4"],
    "2":["5","6"],
    "5":["9","10"],
    "3":[],
    "4":["7","8"],

    "6":[],
    "7":["11","12"],
    "8":[],
    "9":[],
    "10":[],
    "11":[],
    "12":[]
}

visited= []
queue= []
def mybfs(visited,myGraph,starting_node):

        visited.append(starting_node)
        queue.append(starting_node)

        while queue:
            a=queue.pop(0)
            if a=="12":
                break
            else:
                print(a,end=" ")
                for neighbors in myGraph[a]:
                    if neighbors not in visited:
                        visited.append(neighbors)
                        queue.append(neighbors)




mybfs(visited,myGraph,"1")
```

```
1 2 3 4 5 6 7 8 9 10 11
```

DFS:

Snapshot:

```
myGraph={
    "1":["2","3","4"],
    "2":["5","6"],
    "5":["9","10"],
    "3":[],
    "4":["7","8"],

    "6":[],
    "7":["11","12"],
    "8":[],
    "9":[],
    "10":[],
    "11":[],
    "12":[]
}

visited= []

def mydfs(visited,myGraph,starting_node):

    if starting_node not in visited:

            print(starting_node,end=" ")
            if  starting_node=="11":
                print("")
            else:
                visited.append(starting_node)

            for neighbor in myGraph[starting_node]:


                mydfs(visited,myGraph,neighbor)

mydfs(visited,myGraph,"1")
```

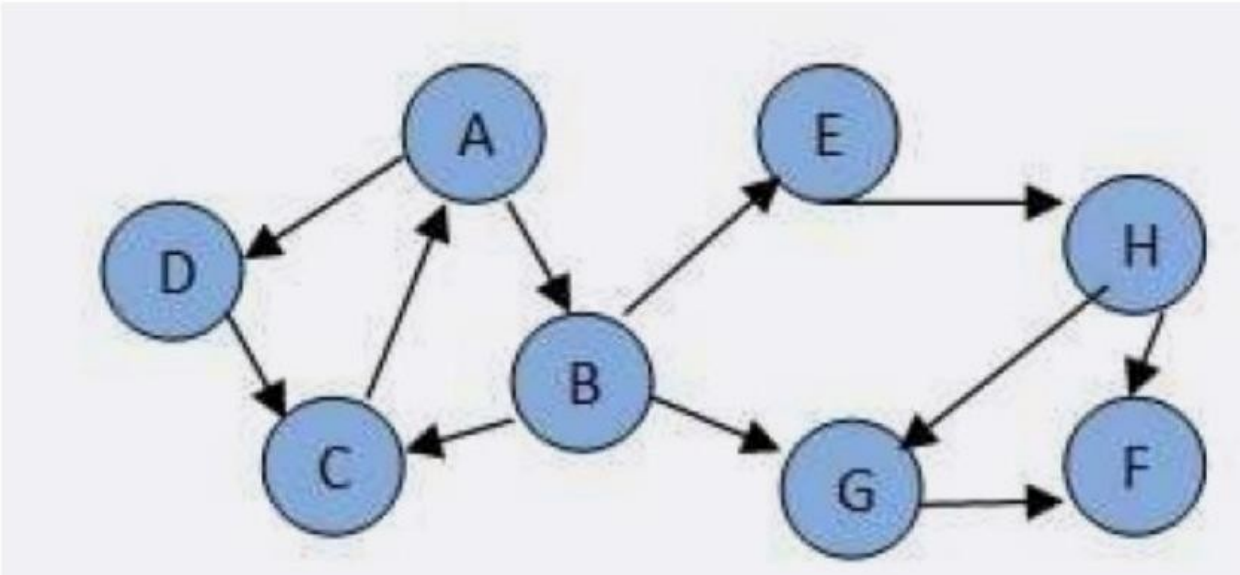
```
1 2 5 9 10 6 3 4 7 11
12 8
```

Task 2:

Write a python program to implement DFS and BFS on the graph given below. Input should be taken from the user.

Start state: A, Goal state: G.

When the program reaches the Goal state, the traversal should stop and prints the path.



BFS:

Snapshot:

```
mygraph ={
    'A' : ['D','B'],
    'B' : ['C','G','E'],
    'C' : ['A'],
    'D' : ['C'],
    'E' : ['H'],
    'F' : [],
    'G' : ['F'],
    'H' : ['G','F'],

}

visited = []
queue = []

def mybfs(mygraph,visited,startState):
    visited.append(startState)
    queue.append(startState)

    while queue:
        a = queue.pop(0)
        print(a, end= " ")
        if a=="G":
            break
        else:
            for neighbour in mygraph[a]:
                if neighbour not in visited:
                    visited.append(neighbour)
                    queue.append(neighbour)

print("Breadth first search traversal:")

mybfs(mygraph,visited,"A")
```

```
Breadth first search traversal:
A D B C G
```

DFS:

Snapshot:

```
mygraph ={
    'A' : ['D','B'],
    'B' : ['C','G','E'],
    'C' : ['A'],
    'D' : ['C'],
    'E' : ['H'],
    'F' : [],
    'G' : ['F'],
    'H' : ['G','F'],

}
visited = []


def mydfs(mygraph,visited,starting_node):
    if starting_node not in visited:
        print(starting_node, end=" ")
        if starting_node=="G":
            print("")
        else:
            visited.append(starting_node)


        for neighbour in mygraph[starting_node]:
            mydfs(mygraph,visited,neighbour)

print("Depth first search traversal:")

mydfs(mygraph,visited,"A")
```

```
Depth first search traversal:
A D C B G
F E H G
```